

LAB 1: Whitted Ray Tracing

Advanced Computer Graphics

Guillem Garcia Sansa 266776 | Jofre Geli de Fuenmayor 266970

1. Task 1: Painting the Image (0.5 points)

This first task's goal was to familiarize ourselves with the rendering loop and how to manipulate the values of pixels stored in the frame buffer. Using the lab handout's Eq. (1), the color of each pixel was determined by its normalized screen coordinates.

$$color = RGBColor(\frac{x}{width}, \frac{y}{height}, 0)$$

To accomplish this, we altered the PaintImage() function in main.cpp. The rendering loop computes the normalized coordinates, iterates through each image's rows (lin) and columns (col), and saves the corresponding RGB value in the frame:

```
void PaintImage(Film* film)
{
    unsigned int sizeBar = 40;

    size_t resX = film->getWidth();
    size_t resY = film->getHeight();

    // Main Image Loop
    for (size_t lin = 0; lin < resY; lin++)
    {
        // Show progression
        if (lin % (resY / sizeBar) == 0)
            std::cout << ".";

        for (size_t col = 0; col < resX; col++)
        {
            //CHANGE...()
            Vector3D random_color = Vector3D((float)col/(float)resX, (float)lin/(float)resY, 0.0);
            film->setPixelValue(col,lin, random_color);
        }
    }
}
```

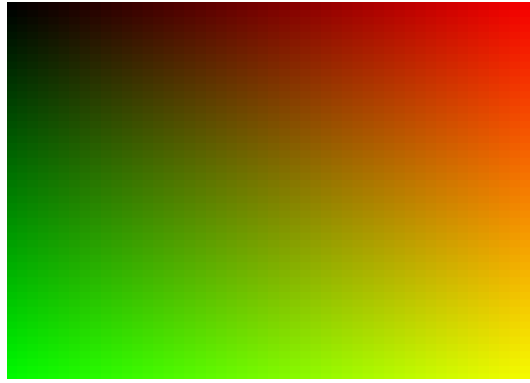


Figure 1: Task 1 output

2. Task 2: Implement the Intersection Integrator (0.5 points)

The objective of this task was to produce an image that indicates which pixels correspond to rays that intersect any geometry in the scene, thereby creating a basic ray-object visibility test. This behaviour was implemented within the IntersectionShader class (IntersectionShader.cpp), which overrides the computeColor() function and inherits from Shader.

The function calls the intersection utility of the framework and sends a ray from the camera through each pixel: `Utils::hasIntersection(r,objList)`. The pixel color is changed to the predefined hitColor (green in our case) if the function returns true. If not, black is used as the background color.

The pertinent code is:

```
Vector3D IntersectionShader::computeColor(const Ray &r, const std::vector<Shape*> &objList, const
std::vector<LightSource*> &lsList) const
{
    //(FILL..)
    if (Utils::hasIntersection(r, objList))
        return hitColor;
    else
        return bgColor;
}
```

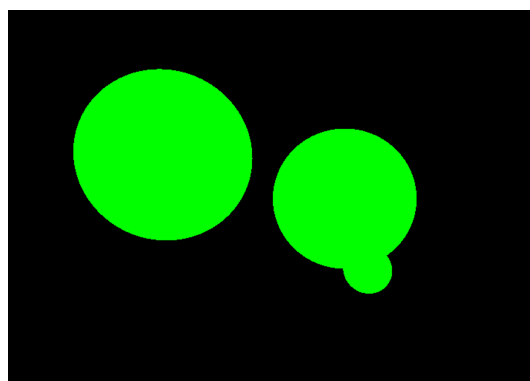


Figure 2: Simple Intersection Shader

3. Task 3: Implement the Depth Integrator (0.5 points)

In this task, we want to visualize the distance between the camera and each ray's initial intersection point in the scene. We mapped the intersection distance to a grayscale value rather than a binary hit color, making objects closer to us appear brighter and objects farther away appear darker. This aids in confirming that the geometry depth ordering and intersection distances are accurate. We use Equation (2) from the lab handout to calculate the depth color:

$$c_i = \max\left(1 - \frac{\text{hitDistance}}{\text{maxDistance}}, 0\right)$$

where c_i is each RGB channel's intensity, hitDistance is the separation between the intersection point and the ray origin, and the maximum visible range is defined by the constant maxDistance .

We implemented `computeColor()` method in a new shader class called `DepthShader`, which is derived from `Shader` using the intersection utilities in the framework. We did it by:

1. Find the closest hit point and distance for each camera ray, use `Utils::getClosestIntersection(r, objList, its)`.
2. Use the above equation to normalize the distance to the $[0,1]$ range.
3. Create a grayscale color, apply this scalar value to each of the three RGB channels.
4. Return the background color (black) if the ray hits nothing.

```
Vector3D DepthShader::computeColor(const Ray &r, const std::vector<Shape*> &objList, const
std::vector<LightSource*> &lsList) const
{
    //(FILL...)
    Intersection its;
    if (Utils::getClosestIntersection(r, objList, its)) {
        Vector3D origin = r.o;
        Vector3D dest = its.itsPoint;
        float dist = (origin - dest).length();
        float dist_n = dist / maxDist;

        return Vector3D(0, 1 - dist_n, 0);
    }

    //else...

    return bgColor;
}
```

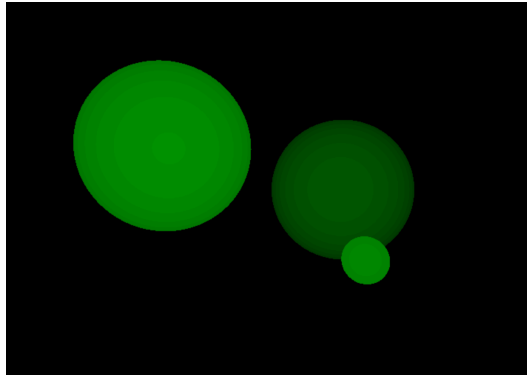


Figure 3: Depth Integrator

4. Task 4: Implement the Normal Integrator (0.5 points)

This task's goal was to display the scene's objects' surface normals as RGB colors. In order to compute illumination in the following Whitted Integrator, it is essential to know the local surface orientation at the intersection point, which is provided by each normal vector. We can verify that each pixel's geometry and normals are computed accurately by mapping the normal components to color values.

Based on the lab handout's Equation (3):

$$color = \frac{normal + (1.0, 1.0, 1.0)}{2}$$

The normal components, which were initially in the range $[-1,1]$ $[-1,1]$, are transformed into the normalized color range $[0,1]$ $[0,1]$ for display by this equation.

Derived from the base Shader class, we created a new shader class called NormalIntegrator. The computeColor() method uses the framework's Utils::getClosestIntersection() function to find the nearest hit point and retrieve its normal. Equation (3) is then used to normalize that normal and convert it to RGB space.

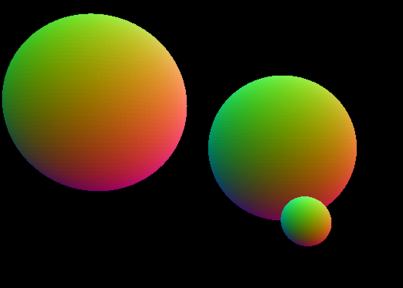
```

Vector3D NormalIntegrator::computeColor(const Ray& r,
    const std::vector<Shape*>& objList,
    const std::vector<LightSource*>& lsList) const
{
    Intersection its;
    // Check for intersection and get intersection info
    if (Utils::getClosestIntersection(r, objList, its))
    {
        // Get normal and normalize
        Vector3D n = its.normal.normalized();

        // Map normal from [-1,1] to [0,1]
        Vector3D color = (n + Vector3D(1.0, 1.0, 1.0)) * 0.5;

        return color;
    }
    else
    {
        return bgColor;
    }
}

```



```
Vector3D Phong::getReflectance(const Vector3D& n,  
    const Vector3D& wo,  
    const Vector3D& wi) const  
{
```

```

    Vector3D n_normalized = n.normalized(); //We normalize the normal vector
    Vector3D wo_normalized = wo.normalized(); //We normalize the wo vector
    Vector3D wi_normalized = wi.normalized(); //We normalize the wi vector

    // Compute: wr = 2(n·wi)n - wi
    Vector3D wr = (n_normalized * (2.0 * dot(n_normalized, wi_normalized))) - wi_normalized; //We
compute the wr with the formula given in theory

    // We compute pd / pi
    Vector3D equation_1 = pd * (1.0 / 3.14159265); //We compute the first part of the equation

    // The final part of the equation will be = Ks * (wo·wr)^α
    double dot_product = dot(wo_normalized, wr); //We compute the (wo·wr)
    Vector3D equation_2 = Ks * pow(dot_product, (double)alpha); //We use the dot_product result
to compute the second part of the equation ( Ks * (wo·wr)^α)

    return equation_1 + equation_2; //We return the sum of both parts of the equation. This
will be the BRDF of the Phong material
}

```

The Whitted integrator used this BRDF for materials with hasDiffuseOrGlossy() = true.

3.5.2 Direct Illumination with Point Lights (2 points)

We want to calculate the local lighting term brought on by visible point-light sources using Equations (6) and (7):

$$L_o(x, \omega_o) = \sum_{s=1}^{nL} L_i^s f_r(\mathbf{n}_x, \omega_i^s, \omega_o) (\omega_i \cdot \mathbf{n}_x) V_s(x) \quad , \quad L_o(x, \omega_o) = a_t \rho_d + \sum_{s=1}^{nL} L_i^s f_r(\mathbf{n}_x, \omega_i^s, \omega_o) (\omega_i \cdot \mathbf{n}_x) V_s(x)$$

To do it, for every source of visible light:

1. Determine the direction ω_i from x to the light.
2. Check visibility $V_s(x)$ casting a shadow ray using `Utils::hasIntersection()`.
3. Add up the light contribution based on the Phong BRDF if it is visible.

```

if (material.hasDiffuseOrGlossy() && ambient > 0.0) {
    final_color += ambient * material.getDiffuseReflectance(); //We do this to avoid having a too
dark scene
    // It is added at the start of the equation.
}

//Exercise 4.5.2
if (material.hasDiffuseOrGlossy()) {
    for (const LightSource* light : lsList) { //We iterate for each light in the scene
        const Vector3D lp = light->sampleLightPosition(); //We get the position of the light
        const Vector3D li = light->getIntensity(); //We get the intensity of the light.
    }
}

```

```

        // The Li will be the first term that we will add to the equation.

        Vector3D L = lp - hit.itsPoint; //We get the vector from the intersection point to
the light source
        const double dist = L.length(); //We get the distance from the intersection point
to the light source
        if (dist <= 0.0) continue; // avoid division by zero

        const Vector3D wi = L / dist; //We get the direction from the intersection point to
the light source

        const Vector3D shadowOrigin = hit.itsPoint + hit_normal * Epsilon; // Small offset
to avoid self-intersection
        Ray shadowRay(shadowOrigin, wi, 0, Epsilon, dist - Epsilon);
        if (Utils::hasIntersection(shadowRay, objList)) {
            continue; // We have to make sure that the light is visible from the
intersection point
        }

        const Vector3D fr = material.getReflectance(hit_normal, wo, wi); //We use the function
created in the phong material to get the BRDF.
        //The fr will be necessary for the function

        const double final_part_equation = dot(hit_normal, wi); //We get the last term of
the equation.

        final_color += Li * fr * final_part_equation; //We finally get the final color by
adding all the terms of the equation.
    }
}

```

This implementation reproduces both diffuse and specular highlights.

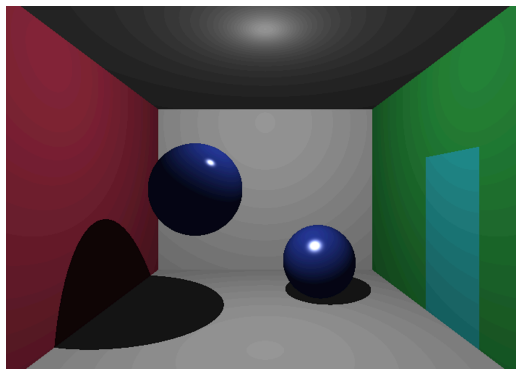


Figure 6: Phong Material Spheres

3.5.3 Mirror Material (2 points)

We want to add perfect specular reflection based on the pseudocode of Algorithm 1 in the lab PDF. The color that a mirror emits is equal to the light that comes in from the direction that it reflects $-\omega_r$. Using `hasSpecular() = true`, we constructed the Mirror material class. A reflection ray is produced recursively in the Whitted integrator if the intersecting material is mirror:

```

    if (material.hasSpecular()) { //This is done for the mirror material
        const Vector3D wr = (r.d - 2.0 * dot(r.d, hit_normal) * hit_normal).normalized(); //We get the reflection direction
        Ray reflRay(hit.itsPoint + hit_normal * Epsilon, wr, r.depth + 1); //We create the reflection ray with a small offset to avoid problems
        final_color += computeColor(reflRay, objList, lsList); //We call the function recursively to get the color of the reflection ray
    }

```

This recursion correctly produces mirror-like reflections of surrounding objects.

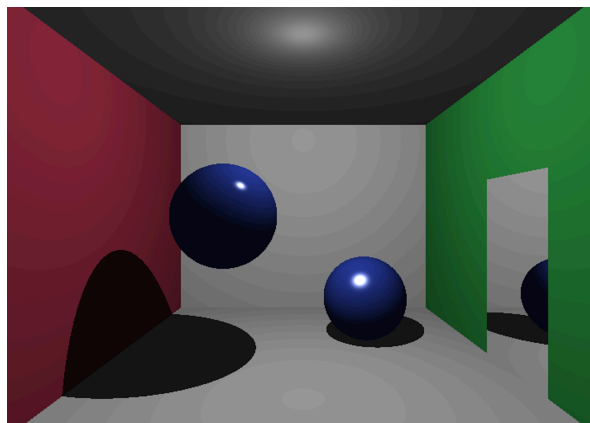


Figure 7: Phong Material Spheres + Mirror

3.5.4 Transmissive Material (3 points)

We want to simulate specular refraction using Algorithm 2's pseudocode and the lab's Eq. (8).

The transmitted ray's direction, w_t is:

$$\omega_t = -\mu_t \omega_o + \mathbf{n}_x \left(\mu_t (\mathbf{n}_x \cdot \omega_o) - \sqrt{1 - \mu_t^2 (1 - (\mathbf{n}_x \cdot \omega_o)^2)} \right),$$

where $\mu_t = \eta_1 / \eta_2$

Total internal reflection takes place, and the ray is reflected rather than refracted if the term inside the square root is negative. We implemented it by creating a Transmissive material with `hasTransmission() = true` and an index of refraction $\eta \approx 1.5$. A refraction ray is calculated and traced recursively in the integrator:

```

    if (material.hasTransmission()) {
        double index_of_refraction = material.getIndexOfRefraction();
        Vector3D hit_normal_local = hit_normal; //We create a local normal to be able to invert it if

```

```

we are exiting the material
    double dot_n_wo = dot(hit_normal, wo); //We compute in advance the dot product between the
normal and wo. (Will be used later for the equation)

    if (dot_n_wo < 0.0) { //We do this to know if we are entering or exiting the material
        hit_normal_local = -hit_normal_local; //We invert the normal if we are exiting the material
        index_of_refraction = 1.0 / index_of_refraction; //We invert the index of refraction if we
are exiting the material
        dot_n_wo = -dot_n_wo; //We also invert the dot product
    }

    const double inside_square_root = 1.0 - index_of_refraction * index_of_refraction * (1.0 -
dot_n_wo * dot_n_wo); //We compute the part that is inside the square root
    if (inside_square_root >= 0.0) { // We check if we have total internal reflection
        // If we are inside the if, means that we don't have total internal reflection

        const Vector3D wt = (-index_of_refraction * wo + (index_of_refraction * dot_n_wo -
std::sqrt(inside_square_root)) * hit_normal_local).normalized(); //We compute wt using the formula
given in theory
        // Pequeño desplazamiento hacia el interior para evitar auto-intersección
        Ray refrRay(hit.itsPoint - hit_normal_local * Epsilon, wt, r.depth + 1); //As shown in the
Pseudo-code, we will create a new ray for the refraction with the computed wt
        final_color += computeColor(refrRay, objList, lsList); //We send the ray again as a
recursive function
    }
    else {
        // If we are inside the else, means that we have total internal reflection

        // If we have total internal reflection, we will trat it as a mirror reflection, this means
that we can directly copy the code used for the mirror.

        //----- MIRROR CODE -----
        const Vector3D wr = (r.d - 2.0 * dot(r.d, hit_normal) * hit_normal).normalized();
        Ray reflRay(hit.itsPoint + hit_normal * Epsilon, wr, r.depth + 1);
        final_color += computeColor(reflRay, objList, lsList);
        //----- END MIRROR CODE -----
    }
}

```

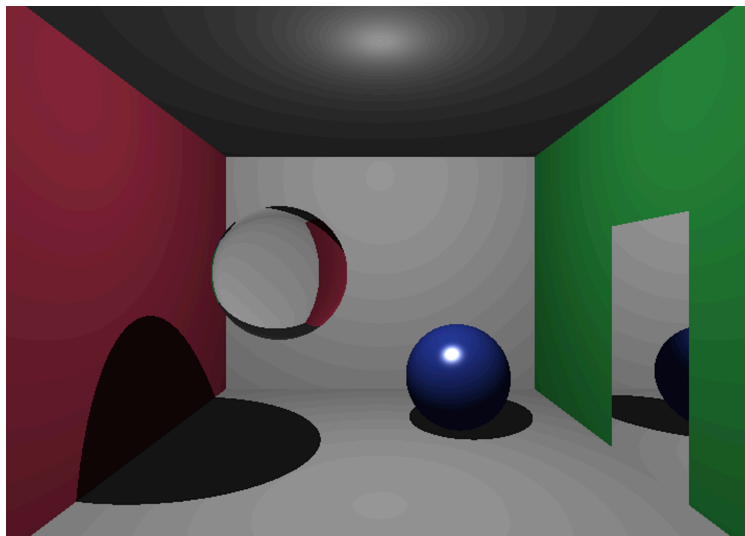


Figure 8: Final Result

Extra: Experimenting

While doing this lab, we were interested in experimenting a bit with the options that we had at the moment of creating the scene and defining the materials of each object. In this section of the report, we are gonna show some of the experiments (outside the requirements of the tasks) that we did to understand on a better way how the code worked.

Experiment 1: Increasing the number of lights

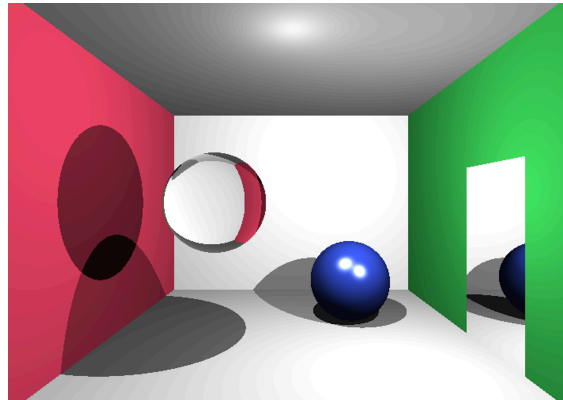


Figure 9: Two light sources

Modified code:

```
PointLightSource* myPointLight1 = new PointLightSource(Vector3D(0, 2.5, 3.0), Vector3D(2.0));  
  
PointLightSource* myPointLight2 = new PointLightSource(Vector3D(2.5, 0, 3.0), Vector3D(1,1,1));  
myScene.AddPointLight(myPointLight1);  
myScene.AddPointLight(myPointLight2);
```

Experiment 2: Playing with light colors

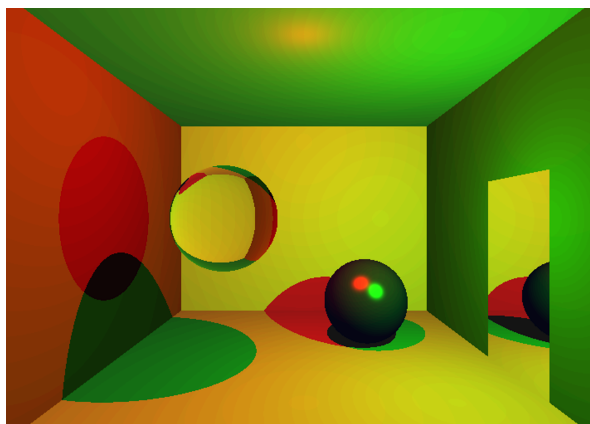


Figure 10: Two light sources with different colors

Modified code:

```
PointLightSource* myPointLight1 = new PointLightSource(Vector3D(0, 2.5, 3.0), Vector3D(3,0,0));  
  
PointLightSource* myPointLight2 = new PointLightSource(Vector3D(2.5, 0, 3.0), Vector3D(0,3,0));  
myScene.AddPointLight(myPointLight1);  
myScene.AddPointLight(myPointLight2);
```

Experiment 3: Changing materials

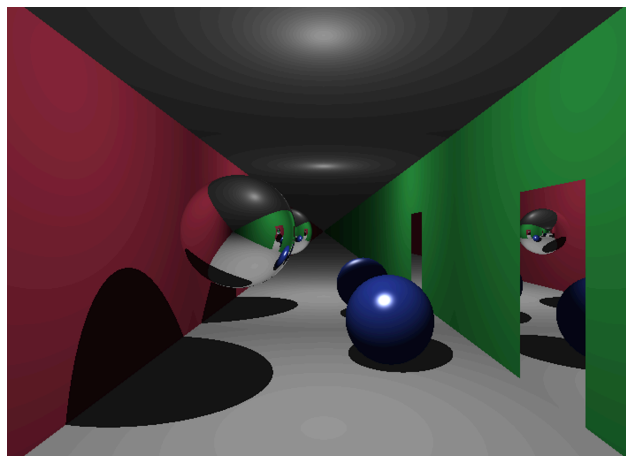


Figure 11: Mirror Sphere and back wall

Modified Code:

```
Shape* backPlan = new InfinitePlan(Vector3D(0, 0, 3 * offset), Vector3D(0, 0, -1), mirror);  
Shape* s2 = new Sphere(radius, sphereTransform2, mirror);
```

Experiment 4: Adding some new elements in the scene

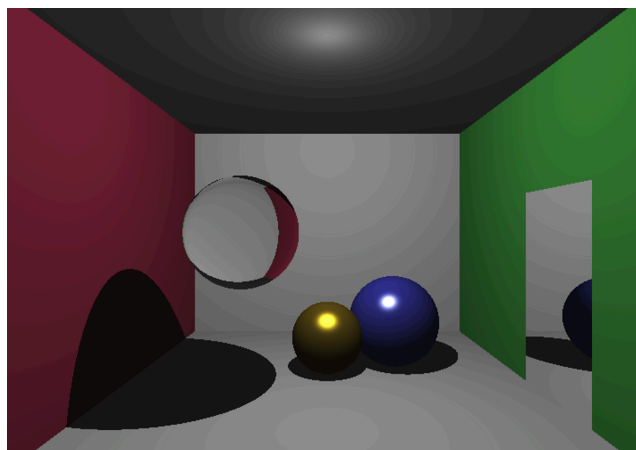


Figure 12: 3 Spheres

Modified Code:

```
Material* gold = new Phong( Vector3D(0.55f, 0.43f, 0.10f), Vector3D(1.00f, 0.86f, 0.23f), 10.0f
); // We define the new material gold.
Shape* new_object = new Sphere(0.75, Matrix4x4::translate(Vector3D(0.0, -offset + 0.75,
5.5)), gold); // We create a new sphere with the gold material.

myScene.AddObject(new_object); // We add the new object to the scene.
```
